HEALTHCARE PLATFORM

Multi-Tenant Dynamic Form System

Architecture · ABDM Compliance · Integrations · Phase-wise Build Guide

GitHub: **https://github.com/nwoow/healthcare-platform**

1. Project Vision & Goals

A production-grade, multi-tenant Healthcare Management Platform. Each hospital or diagnostic centre is an isolated tenant. Admins design dynamic clinical forms. Clinicians fill them with full conditional logic and patient history awareness. Every action flows through Kafka into audit logs, third-party integrations, and analytics.

What We Are Building

- Multi-tenant SaaS — each hospital is an isolated tenant with zero data leakage
- Dynamic form builder — admin designs specialty-specific forms visually, no code required
- RBAC + ABAC via dedicated IAM microservice — CASL policy engine with Redis caching
- Kafka-driven event architecture — all services communicate asynchronously
- Hybrid data: PostgreSQL for relational, MongoDB for dynamic forms and submissions
- ABDM compliance architecture — ABHA identity, FHIR R4, consent management built in
- Third-party integration framework — webhook delivery to any external system
- Field-level audit trail — PHI masking, compliance reporting, immutable logs

Core Design Principles

- Tenant isolation — tenantId in every DB row and JWT, enforced at every layer
- All access decisions through IAM Service — no inline permission checks in business logic
- Kafka partitioned by tenant_id — ordered per tenant, parallel across tenants
- Forms are data not code — JSON schema in MongoDB, rendered dynamically
- Audit trail at field level — every data operation logged with PHI masking
- ABDM-ready architecture — FHIR R4 layer and consent management built today, NHA registration activates M2/M3

2. Technology Stack

Layer	Technology
Frontend	Next.js 15 (App Router), Material UI, React Hook Form, SWR, recharts
Form Builder	dnd-kit drag-and-drop, custom JSON schema engine
Backend	NestJS (8 microservices), Passport.js, CASL for RBAC + ABAC
Auth	JWT httpOnly cookies, refresh token rotation, bcrypt
PostgreSQL ORM	Prisma — type-safe queries, migrations, schema management
MongoDB ODM	Mongoose — schema validation on writes, typed models
Primary DB	PostgreSQL — users, roles, tenants, patients, audit logs
Document DB	MongoDB — form schemas, submissions, workflows, integrations
Cache	Redis — permission cache 60s TTL, event-driven invalidation
Message Broker	Apache Kafka — 9 topics, partitioned by tenant_id

File Storage	MinIO (local) / AWS S3 (production)
Queue	BullMQ (Redis-backed) — async jobs, PDF generation
Containers	Docker Compose (local) → single Linux VM (production)
API Gateway	Kong — tenant routing, rate limiting, SSL termination
API Docs	Swagger UI on all 8 services with example request bodies
Monitoring	Prometheus + Grafana, Kafka consumer lag, pod health

3. Microservices Architecture

Service Inventory — Current State

Service	Port	Status / Key Responsibility
Auth Service	3001	✓ Login, JWT, refresh tokens, user management, super admin seed
IAM Service	3002	✓ RBAC + ABAC, CASL policy eval, Redis cache, Kafka invalidation
Form Builder	3003	✓ Form schemas, versioning, publish workflow, Kafka publisher
Submission Service	3004	✓ Fill/draft/submit/approve/reject, audit trail, analytics
Patient Service	3006	✓ Registry, MRN, ABHA, FHIR R4, consent, history flags
Audit Service	3008	✓ Append-only logs, 9 Kafka consumers, compliance report
Integration Service	3009	✓ NEW — webhooks, all-topic Kafka consumer, retry + logs
Tenant Service	3010	✓ Hospital onboarding, ABDM config, provisioning logs
Workflow Service	3005	⏳ Pending — Phase 3
Notification Service	3007	⏳ Pending — Phase 3

Communication Model

Synchronous — REST / gRPC

- API Gateway → IAM Service: permission check on every request (gRPC, latency-critical, <1ms with Redis cache)
- Any Service → Auth Service: JWT token validation
- Form Renderer → Patient Service: fetch history flags for conditional logic

Asynchronous — Kafka (9 Topics)

Topic	Producer	Consumers
tenant.provisioned	Tenant	Audit, Integration
auth.login.success	Auth	Audit, Integration
auth.login.failed	Auth	Audit, Integration

iam.policy.updated	IAM	IAM (cache flush), Audit, Integration
form.published	Form Builder	Audit, Integration
submission.submitted	Submission	Audit, Integration
patient.history.updated	Patient	Audit, Integration
patient.abha.linked	Patient	Audit, Integration
workflow.started	Workflow (future)	Audit, Integration

Partition key = tenant_id on all topics. All events for one tenant stay on the same partition — guaranteed ordering. Different tenants processed in parallel — horizontal throughput with zero cross-tenant interference.

4. IAM Service — RBAC + ABAC

Standalone NestJS microservice. Every other service calls it via gRPC before processing any request. Redis caches decisions for 60 seconds. Kafka event invalidates cache on any role or permission change.

System Roles

Role	Scope & Default Access
super_admin	Platform-wide — bypasses all ABAC checks, manages all tenants
tenant_admin	Own hospital — manage forms, users, submissions, departments
department_admin	Own department — manage forms and users in their dept
doctor	Own patients — read/submit forms, view assigned patient submissions
nurse	Nursing forms — read+submit nursing-tagged forms
receptionist	Registration only — create patients, read appointments
diagnostics_staff	Lab and radiology forms only
auditor	Read-only — view all submissions and audit logs, no writes

ABAC Attribute Rules

Attribute Key	Rule Example
department	Gastro doctor sees only gastroenterology forms
specialisation	Skin specialist sees dermatology forms only
patient_assignment	Doctor can only submit forms for their assigned patients
grade	Only consultants and above can approve submissions
shift	Night shift nurses see night-specific form set
working_hours	Forms accessible only during configured working hours

Policy Evaluation Flow

1. Request arrives → Service calls IAM gRPC /evaluate with userId, action, resource, resourceId, tenantId
2. Check Redis cache key: perm:{tenantId}:{userId}:{action}:{resource} — return immediately if hit (0.3ms)
3. Cache miss: load user roles from PostgreSQL
4. RBAC check — does role have permission for this action on this resource?
5. ABAC check — do user attributes match resource policy conditions?
6. super_admin role bypasses ABAC entirely — early return allowed:true
7. Cache result with 60s TTL
8. Return { allowed: boolean, reason: string, cached: boolean }

5. ABDM Compliance Architecture

The Ayushman Bharat Digital Mission (ABDM) is India's national digital health infrastructure. ABDM certification is becoming mandatory for AB-PMJAY hospitals in 2026. The platform's architecture is ABDM-ready today — Milestone 1 is fully built. M2 and M3 activate with NHA registration.

ABDM Milestones

Milestone	What It Means — Status
M1 — ABHA Identity	ABHA number capture, verification, and patient linking ✓ BUILT
M1 — FHIR R4 Layer	HL7 FHIR R4 resource generation for all clinical data ✓ BUILT
M1 — Consent Management	Patient consent lifecycle (request/grant/revoke/expire) ✓ BUILT
M1 — ABDM Tenant Config	Per-tenant HFR, HIP, HIU IDs and sandbox/production mode ✓ BUILT
M2 — HIP Data Sharing	Share digital records via patient ABHA app — needs NHA client_id
M3 — HIU Data Access	Request records from other facilities with consent — needs NHA cert

What Is Built Today

ABHA Identity Fields (Patient Service)

- abhaNumber — 14-digit unique national health ID, validated format /^\d{14}$/, unique constraint
- abhaAddress — human-readable ID in format name@abdm, validated
- POST /patients/:id/link-abha — validates, sets abhaVerified=true, publishes patient.abha.linked Kafka event
- GET /patients/:id/abha-status — returns masked ABHA (first 6 digits only) + verification status

FHIR R4 Service (Patient Service)

- patientToFhir(patient) → FHIR R4 Patient resource with MRN + ABHA identifiers, demographics, blood group extension
- submissionToFhir(submission, schema) → FHIR Bundle of Observations — each form field maps to an Observation

- consultationToFhir(submission, patient) → FHIR Composition with SNOMED code 371530004 (OPD consultation)
- Live endpoints: GET /patients/:id/fhir and GET /patients/:id/fhir/observations

Consent Management (Patient Service)

- ConsentArtifact schema — consentId, patientId, purpose, dataCategories, dateRange, status lifecycle
- Purposes: CARE_MANAGEMENT | BREAK_THE_GLASS | PAYMENT | RESEARCH
- Status flow: REQUESTED → GRANTED → REVOKED / EXPIRED
- Data categories: OPConsultation, DiagnosticReport, DischargeSummary, Prescription, ImmunizationRecord
- Demo mode: auto-grants after 2 seconds — replace with real ABDM Gateway for production

Per-Tenant ABDM Configuration (Tenant Service)

- abdmEnabled, abdmFacilityId (HFR), abdmHipId, abdmHiuId, abdmSandboxMode (default: true)
- PATCH /tenants/:id/abdm-config — update any config field
- GET /tenants/:id/abdm-status — returns config + patientsWithAbha count + linkageRate %

Path to Full Certification

9. Register facility at https://facility.ndhm.gov.in (Health Facility Registry)
10. Apply for HIP credentials at ABDM sandbox: https://sandbox.abdm.gov.in
11. Complete M1 functional testing with NHA-empanelled agency (FIME India / Suma Soft)
12. Replace abdmSandboxMode: true with false and populate real HIP/HIU IDs in tenant config
13. Implement HIP callback endpoint /patients/hip/on-discover for M2 data sharing
14. Implement HIU consent notification webhook for M3 cross-facility access

M2 and M3 are configuration and API integration work. All data models, FHIR conversion, and consent flows are already in place.

6. Third-Party Integration Framework

A dedicated Integration Service (port 3009) allows any tenant to connect external systems — lab software, pharmacy, insurance, ERP, WhatsApp gateways — to receive real-time events from the platform via configurable webhooks.

How It Works

15. Admin creates an integration in /admin/integrations with webhook URL and auth config
16. Admin selects which Kafka topics trigger this integration (e.g. submission.submitted)
17. Integration Service Kafka consumer receives every event on all 9 topics
18. Matches event to active integrations for that tenant with matching trigger
19. Builds payload from template — replaces {{fieldId}} with actual submitted values
20. POSTs to external webhook URL with configured authentication
21. Retries up to 3 times with exponential backoff on failure (1s / 2s / 3s)
22. Saves IntegrationLog with HTTP status, duration, request and response
23. Updates stats: totalCalled, totalSuccess, totalFailed, lastCalledAt

Integration Types Supported

Type	Example Use Case
WEBHOOK	Any HTTP endpoint — most common, send JSON payload on event
REST_API	Structured REST call with custom headers and payload shape
FHIR_ENDPOINT	Send FHIR R4 Bundle to external FHIR server
LAB_SYSTEM	Notify lab software when diagnostic form is submitted
PHARMACY	Send prescription data to pharmacy management system
INSURANCE	Trigger insurance claim on form approval
ERP	Sync patient visit data to hospital ERP
CUSTOM	Any external system with custom payload template

Authentication Methods

- None — open endpoints
- Bearer Token — Authorization: Bearer {token} header
- API Key — custom header name and value
- Basic Auth — Base64 encoded username:password

Reliability Features

- Retry up to 3 attempts with 1s / 2s / 3s exponential backoff
- Per-call IntegrationLog — HTTP status, response body, duration, success flag
- Success rate tracking — totalSuccess / totalCalled shown as % on UI
- Auto-marks integration status=error if all retries fail
- Test Connection endpoint — POST /integrations/:id/test fires a test payload and returns result
- Retry failed button per log entry in the logs UI

API Endpoints (port 3009)

Endpoint	Description
GET /integrations?tenantId=X	List all integrations for tenant
POST /integrations	Create new integration with config and triggers
PUT /integrations/:id	Update config, triggers, or auth
DELETE /integrations/:id	Soft delete (status=deleted)
PATCH /integrations/:id/toggle	Toggle active ↔ inactive
POST /integrations/:id/test	Fire test payload — returns {success, statusCode, durationMs}
GET /integrations/:id/logs	Delivery logs newest first, default limit 50

7. Form Data Audit Trail

Every operation on a clinical form submission is logged at the field level — not just at the service level. This is critical for ABDM compliance, clinical accountability, and regulatory requirements under DPDP Act 2023.

Actions Tracked (FormDataAuditLog)

Action	When Logged
DRAFT_SAVED	POST /submissions with status: draft
FORM_SUBMITTED	POST /submissions with status: submitted
FORM_APPROVED	PATCH /submissions/:id/status → approved
FORM_REJECTED	PATCH /submissions/:id/status → rejected
FORM_VIEWED	GET /submissions/:id (with actorId query param)
FHIR_EXPORTED	GET /submissions/:id/fhir
CONSENT_CHECKED	On approval when patient has ABHA linked

PHI Field Masking

Form fields can be marked as PHI (Protected Health Information) in the form builder by toggling the "PHI Field" switch. When a PHI field value changes or is viewed:

- The audit log stores ***PHI*** instead of the actual value
- The access event is still fully logged — who viewed it, when, from which IP
- Non-PHI fields store actual old and new values for full change tracking
- PHI fields show a red lock icon in the form builder canvas and renderer

Compliance Report

GET /audit/compliance-report?tenantId=X&from=YYYY-MM-DD&to=YYYY-MM-DD returns:

- phiAccessCount — number of times PHI fields were viewed
- fhirExportCount — number of FHIR exports
- consentCheckCount — number of ABDM consent verifications
- approvalCount and rejectionCount
- topActors — top 10 users by data access count
- eventsByDay — daily event counts for the period

The compliance report is available in the Analytics page under the Compliance tab, with a date range picker and CSV export. All data sourced from the append-only audit_logs table — zero updates or deletes ever.

8. Database Design

PostgreSQL Tables (shared, tenantId-isolated)

Table	Service / Contents
users	Auth — email, passwordHash, tenantId, name, isActive
refresh_tokens	Auth — active JWT refresh tokens
roles	IAM — system + tenant-scoped roles
permissions	IAM — resource + action pairs
role_permissions	IAM — role to permission mapping with optional ABAC conditions
user_roles	IAM — user to role assignment per tenant with optional expiry
user_attributes	IAM — ABAC key-value attributes (department, grade, shift)
resource_policies	IAM — ABAC conditions per resource (restrict by attributes)
tenants	Tenant — all registered hospitals
tenant_configs	Tenant — limits, features, ABDM config per hospital
tenant_provisioning_logs	Tenant — 5-step onboarding audit trail
patients	Patient — registry with MRN, ABHA fields, demographics
audit_logs	Audit — immutable append-only Kafka event log

MongoDB Collections

Collection (Database)	Service / Contents
form_schemas (healthcare_forms)	Form Builder — form definitions, sections, fields, conditional logic
form_versions (healthcare_forms)	Form Builder — immutable snapshot on every publish
form_submissions (healthcare_submissions)	Submission — all draft and submitted forms, flexible data shape
form_data_audit_logs (healthcare_submissions)	Submission — field-level audit trail per submission
patient_histories (healthcare_patients)	Patient — visit records, allergies, conditions, medications
consent_artifacts (healthcare_patients)	Patient — ABDM consent lifecycle per patient
integration_configs (healthcare_integrations)	Integration — webhook configs and triggers per tenant
integration_logs (healthcare_integrations)	Integration — immutable delivery log per webhook call

9. Dynamic Form System

Form Layouts — Admin Chooses Per Form

Layout	How It Renders
Wizard	MUI Stepper — one section per page, Next validates current section, progress bar, submit on last step
Accordion	All sections visible, click to expand, completion count per section header
Tabs	Each section is a tab, unfilled required count badge, Previous/Next navigation

Field Types (11 supported)

Field Type	Description
Text	Single line input with character limit
Textarea	Multi-line with character counter
Number	With +/- steppers and unit label
Date	MUI DatePicker with dayjs adapter
Dropdown	MUI Select, Autocomplete if >5 options
Radio	Card-style options — selected shows teal border + background
Checkbox	Same card style as radio, multiple selection
Slider	Teal track, value badge above thumb, min/max labels
File Upload	Drag-and-drop zone, file chips with remove, S3 backed
Signature	HTML canvas with clear button, base64 output
Repeater	Dynamic add/remove rows — medication list, allergy list

Conditional Logic Types

Trigger Type	How It Works
field_value	Show/hide field or section when another field in the current form equals a value
patient_history	Show section when patient history flag is true (prevEndoscopy, chronicConditions, etc.)

Form Versioning

- Every publish creates an immutable FormVersion snapshot — old versions never modified
- Submissions pin to the version they were filled against — schema drift does not affect historical data
- Admin can view version history and diff between any two versions

10. Scalability & Cost Model

Scale Path

Stage	Tenants / Users / Est. Monthly Cost
Pilot	5 hospitals · 500 users · ~₹8,000/month
Growth	50 hospitals · 5,000 users · ~₹40,000/month
Scale	500 hospitals · 50,000 users · ~₹2,00,000/month
Enterprise	5,000+ hospitals · 5,00,000+ users · Custom pricing

Cost Efficiency Decisions

- Shared PostgreSQL with tenantId isolation — 80% lower DB cost vs schema-per-tenant
- MongoDB flexible document model — no schema migrations when form fields change
- Kafka webhook push model — zero polling costs for integrations
- Redis permission cache — 0.3ms vs 5ms per check, 60% reduction in PostgreSQL read load
- Docker Compose single-server — no Kubernetes overhead for pilots and small deployments

PostgreSQL Scaling Path

- Phase 1-2: Single node, tenantId isolation, PgBouncer connection pooling
- Phase 3: Primary + 2 read replicas — IAM reads from replica, analytics from replica
- Phase 4: Shard by tenant_id — routing table in Tenant Service maps tenant to shard
- Phase 5: Citus extension for enterprise tenants with 10,000+ concurrent users

MongoDB Scaling Path

- Phase 1-2: Single 3-node ReplicaSet — Primary writes, Secondary reads for analytics
- Phase 3: Dedicated ReplicaSet for form_submissions (high write volume)
- Phase 4: Enable sharding — shard key tenant_id, 3 shards initially
- Phase 5: Separate clusters for schemas vs submissions vs patient history

Kafka Scaling Path

- Phase 1: Single broker (development only)
- Phase 2: 3-broker cluster, 12 partitions per topic, replication factor 3
- Phase 4: 48 partitions per high-volume topic (submission.submitted)
- Phase 5: Separate Kafka clusters per region for data residency compliance

11. Multi-Tenant Management

The platform has three distinct portals for three distinct user types. All share one Next.js 15 frontend and redirect automatically by role after login.

Portal Structure

Portal	URL / Login / Role
Super Admin	/super-admin/login — superadmin@platform.com / SuperAdmin@123
Tenant Admin	/login — admin@hospital.com / Admin@123
Clinician	/login — doctor or nurse credentials

Hospital Onboarding Flow (3-Step Wizard)

24. Super Admin opens /super-admin/dashboard → clicks "+ Register New Hospital"
25. Step 1: Hospital name, subdomain (availability checked live), plan, region, type
26. Step 2: Admin name and email — system auto-generates secure password [Name][4digits]!
27. Step 3: Review — system runs 5 provisioning steps automatically:
    - create_tenant → create_config → create_admin_user → seed_roles → activate
28. Success screen shows generated password — copy it immediately, shown once only
29. Tenant Admin logs in at /login with generated credentials
30. Tenant Admin invites doctors and nurses via /admin/users

Tenant Isolation

- tenantId is part of every JWT payload — set at login, never changes
- Every API query adds WHERE tenantId = currentUser.tenantId automatically
- MongoDB documents all carry tenantId field — compound indexes enforced
- Super Admin role bypasses tenant isolation — can see all tenants
- No tenant can ever access another tenant's patients, forms, or submissions

12. Phase-wise Implementation Status

PHASE 0 — Foundation ✓ COMPLETE

- Monorepo structure, Docker Compose, all services scaffolded
- PostgreSQL, MongoDB, Redis, Kafka, Kafka UI all running
- Prisma migrations, Mongoose connections verified

PHASE 1 — Prototype ✓ COMPLETE

- Auth Service: JWT login, httpOnly cookies, refresh tokens, seeded admin
- IAM Service: RBAC seeded, CASL policy engine, evaluate endpoint
- Form Builder Service: schema CRUD, versioning, publish, accessible endpoint
- Submission Service: draft/submit, approve/reject, filters
- Frontend: login, admin dashboard, form builder UI, form renderer, submissions view

PHASE 2 — Multi-Tenancy + Kafka + ABAC ✓ COMPLETE

- Tenant Service: 5-step onboarding, admin user auto-creation, ABDM config
- Kafka shared package: KafkaProducerService, KafkaConsumerService, all 9 topics
- ABAC in IAM: user attributes, resource policies, Redis cache, Kafka invalidation
- Patient Service: PostgreSQL registry + MongoDB history, ABHA, FHIR R4, consent
- Audit Service: 9 Kafka consumers, append-only audit_logs, compliance report

PHASE 3 — UI + Tenant Management ✓ COMPLETE

- Material UI throughout with 5 one-click colour themes
- Super Admin portal: /super-admin/login, dashboard, 3-step hospital registration wizard
- Admin portal: MUI layout, form builder with layout picker, patient management
- Clinician portal: hero dashboard, patient selector, multi-layout form renderer
- Analytics: 3-tab dashboard (Submissions, Patients, Compliance)
- In-app notifications: bell icon, 30s polling, toasts

PHASE 4 — ABDM + Integrations + Audit ✓ COMPLETE

- ABHA identity fields, link-abha endpoint, FHIR R4 service
- Consent management — full lifecycle with demo auto-grant
- Integration Service (port 3009) — webhook delivery, all-topic Kafka consumer
- Form data audit trail — field-level, PHI masking, compliance report
- Patient management UI — 5-tab detail page with FHIR viewer and consent tab
- Analytics completion — specialty chart, patient analytics, compliance tab

PHASE 5 — Workflow + Notifications ⏳ PENDING

- Workflow Service (port 3005) — state machine, branching, escalation timers
- Notification Service (port 3007) — email via nodemailer, SMS via Twilio
- Diagnostics page — service health grid, Kafka lag, Redis hit rate

PHASE 6 — Production Readiness ⏳ PENDING

- Nginx reverse proxy + Let's Encrypt SSL
- Environment hardening — secrets management, rate limiting, HTTPS-only cookies
- PostgreSQL daily backup, MongoDB dump, Redis AOF persistence
- NHA production registration for ABDM M2/M3 certification

13. Quick Reference

All Service Ports

Service	Port	URL / Notes
Frontend (Next.js 15)	3000	http://localhost:3000

Auth Service	3001	http://localhost:3001/api/docs
IAM Service	3002	http://localhost:3002/api/docs
Form Builder	3003	http://localhost:3003/api/docs
Submission Service	3004	http://localhost:3004/api/docs
Patient Service	3006	http://localhost:3006/api/docs
Audit Service	3008	http://localhost:3008/api/docs
Integration Service	3009	http://localhost:3009/api/docs — NEW
Tenant Service	3010	http://localhost:3010/api/docs
PostgreSQL	5433	host port (5432 taken by Windows native postgres)
MongoDB	27017	replica set rs0
Redis	6379	permission cache + sessions
Kafka	9092	message broker
Kafka UI	8083	http://localhost:8083 — no login
Mongo Express	8081	http://localhost:8081 — admin/admin123
pgAdmin	8082	http://localhost:8082 — admin@healthcare.com/admin123

Test Credentials

Role	Email / Password / Portal
Super Admin	superadmin@platform.com / SuperAdmin@123 / /super-admin/login
Tenant Admin	admin@hospital.com / Admin@123 / /login
Doctor	Invite via /admin/users — generated credentials
Nurse	Invite via /admin/users — generated credentials

GitHub Repository

https://github.com/nwoow/healthcare-platform

Starting the System

```
# Step 1 — Start Docker
docker-compose up -d

# Step 2 — Start all backend services (each in a separate terminal)
cd services/auth        && pnpm run start:dev   # port 3001
cd services/iam         && pnpm run start:dev   # port 3002
cd services/form-builder && pnpm run start:dev  # port 3003
cd services/submission  && pnpm run start:dev   # port 3004
cd services/patient     && pnpm run start:dev   # port 3006
cd services/audit       && pnpm run start:dev   # port 3008
cd services/integration && pnpm run start:dev   # port 3009
cd services/tenant      && pnpm run start:dev   # port 3010
```

```
# Step 3 — Start frontend
cd apps/frontend && pnpm run dev                # port 3000
```